ΚΛΑΣΕΙΣ - CLASSES

ΣΤΗΝ C++

ΕΙΣΑΓΩΓΗ – ΕΠΑΝΑΛΗΨΗ ΑΠΟ ΤΑ ΠΡΟΗΓΟΥΜΕΝΑ

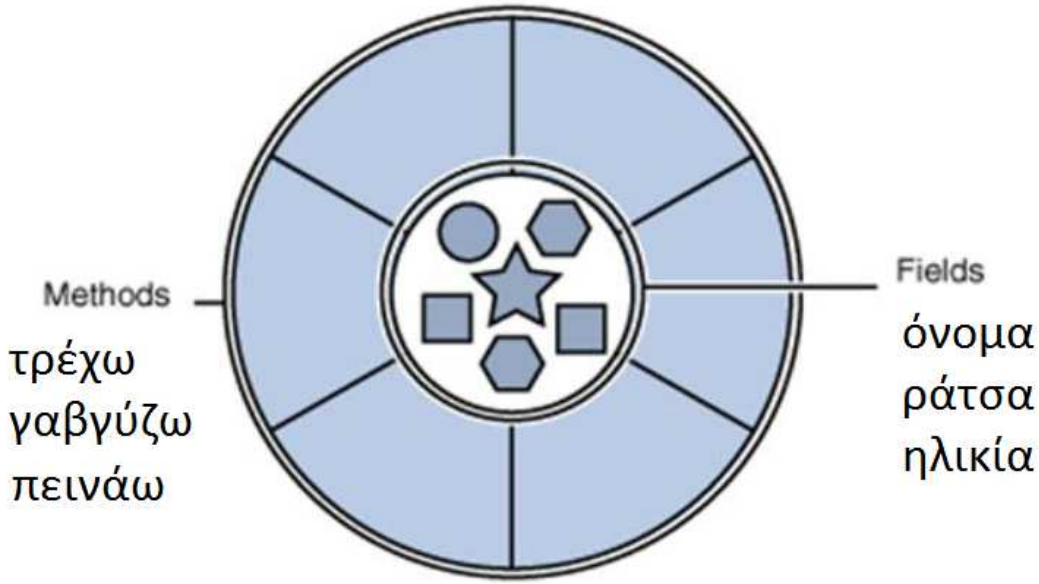
- Η καταννόηση της έννοιας του **αντικειμένου (object)** είναι βασική για την καταννόηση του **αντικειμενοστραφούς προγραμματισμού (object-oriented programming)**.

- Αντικείμενα υπάρχουν παντού στον πραγματικό κόσμο: θρανία, τηλεοράσεις, αυτοκίνητα, κτλ. Στον αντικειμενοστραφή προγραμματισμό ένα αντικείμενο δεν διαφέρει σχεδόν καθόλου με την θεώρηση στον πραγματικό κόσμο

- Κάθε αντικείμενο διαθέτει τα εξής:
  - Τα **πεδία (fields)** ή **χαρακτηριστικά (attributes)** του.
  - Τις **μεθόδους (methods)** του που αποτελούν τις ενέργειες που μπορεί να εκτελέσει.

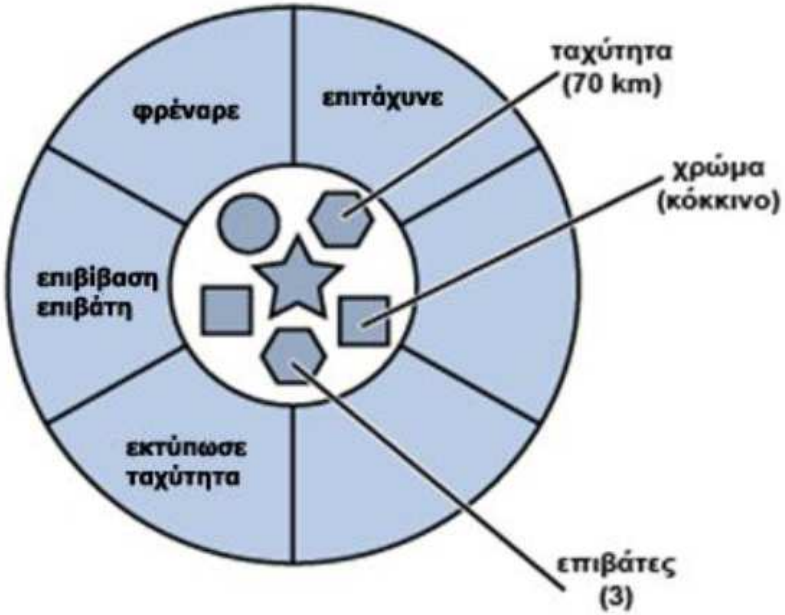
- Για παράδειγμα το αντικείμενο με όνομα «Rocky», το οποίο είναι σκυλάκι, έχει ως πεδία το όνομά του (που είναι «Rocky»), το χρώμα του (άσπρο) και την ηλικία του (5 ετών) ενώ ως μεθόδους τις ενέργειες «γάβγισμα», «τρέξιμο» και «κούνημα ουράς».

- Μπορούμε να φανταστούμε ένα αντικείμενο ως ένα «μπαλάκι» το οποίο έχει στο κέντρο/εσωτερικό του τα πεδία του και γύρω από αυτό τις μεθόδους του.



- Βασική επιδίωξη του αντικειμενοστραφούς προγραμματισμού, είναι η **επικοινωνία** με κάποιο αντικείμενο να γίνεται μόνο μέσω των μεθόδων του κι όχι με άμεση πρόσβαση στα πεδία του.

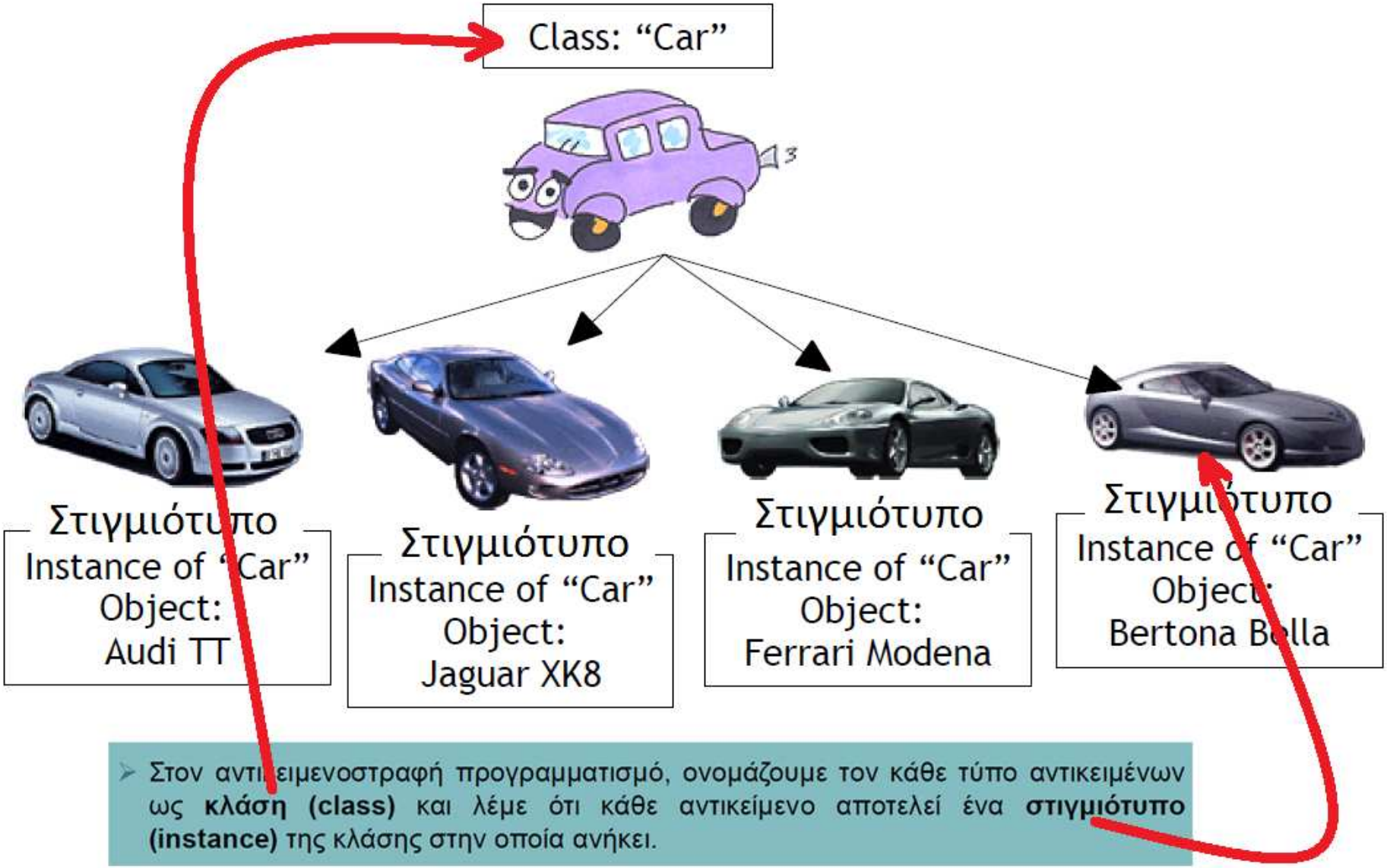
- Με τον όρο **επικοινωνία** αναφερόμαστε στα εξής:
  - όταν ζητάμε από το αντικείμενο να μεταβάλλει τις τιμές κάποιων πεδίων του.
  - όταν ζητάμε από το αντικείμενο να πληροφορηθούμε (π.χ. να μας εκτυπώσει) τις τιμές κάποιων πεδίων του.



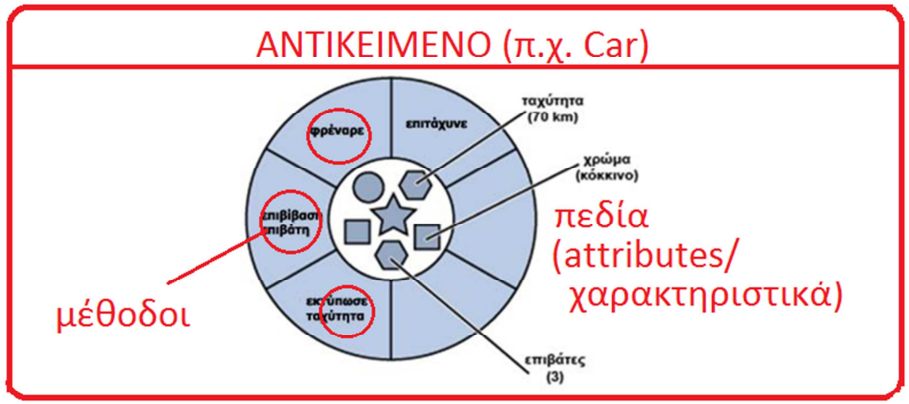
- Στον πραγματικό κόσμο, μπορούμε να βρούμε πολλά αντικείμενα, τα οποία είναι του ίδιου τύπου. Για παράδειγμα, με μία απλή ματιά στον δρόμο του σπιτιού μας μπορούμε να δούμε πολλά αυτοκίνητα, δηλαδή πολλά αντικείμενα του τύπου «Αυτοκίνητο».

- Αυτό που διαφοροποιεί ουσιαστικά το κάθε αντικείμενο ενός τύπου από όλα τα υπόλοιπα, είναι οι τιμές που έχει τα πεδία του (θα μπορούσαν όμως να υπάρξουν και δύο διαφορετικά αντικείμενα με ακριβώς ίδιες τιμές σε όλα τα πεδία τους).

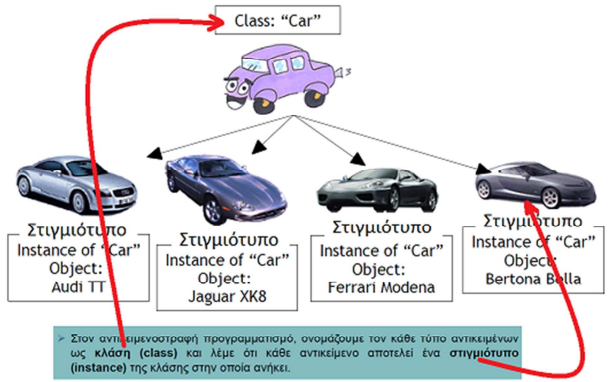
- Στον αντικειμενοστραφή προγραμματισμό, ονομάζουμε τον κάθε τύπο αντικειμένων ως **κλάση (class)** και λέμε ότι κάθε αντικείμενο αποτελεί ένα **στιγμιότυπο (instance)** της κλάσης στην οποία ανήκει.



- Στον αντικειμενοστραφή προγραμματισμό, ονομάζουμε τον κάθε τύπο αντικειμένων ως **κλάση (class)** και λέμε ότι κάθε αντικείμενο αποτελεί ένα **στιγμιότυπο (instance)** της κλάσης στην οποία ανήκει.



κάθε τύπος αντικειμένου ονομάζεται κλάση/Class



- Αντικείμενα υπάρχουν παντού στον πραγματικό κόσμο: θρανία, τηλεοράσεις, αυτοκίνητα, κτλ. Στον αντικειμενοστραφή προγραμματισμό ένα αντικείμενο δεν διαφέρει σχεδόν καθόλου με την θεώρηση στον πραγματικό κόσμο

- Κάθε αντικείμενο διαθέτει τα εξής:
  - Τα **πεδία (fields)** ή **χαρακτηριστικά (attributes)** του.
  - Τις **μεθόδους (methods)** του που αποτελούν τις ενέργειες που μπορεί να εκτελέσει.

- Βασική επιδίωξη του αντικειμενοστραφούς προγραμματισμού, είναι η **επικοινωνία** με κάποιο αντικείμενο να γίνεται μόνο μέσω των μεθόδων του κι όχι με άμεση πρόσβαση στα πεδία του.
- Με τον όρο **επικοινωνία** αναφερόμαστε στα εξής:
  - όταν ζητάμε από το αντικείμενο να μεταβάλλει τις τιμές κάποιων πεδίων του.
  - όταν ζητάμε από το αντικείμενο να πληροφορηθούμε (π.χ. να μας εκτυπώσει) τις τιμές κάποιων πεδίων του.
- Στον πραγματικό κόσμο, μπορούμε να βρούμε πολλά αντικείμενα, τα οποία είναι του ίδιου τύπου. δηλαδή πολλά αντικείμενα του τύπου «Αυτοκίνητο».
- Αυτό που διαφοροποιεί ουσιαστικά το κάθε αντικείμενο ενός τύπου από όλα τα υπόλοιπα, είναι οι τιμές που έχει τα πεδία του (θα μπορούσαν όμως να υπάρξουν και δύο διαφορετικά αντικείμενα με ακριβώς ίδιες τιμές σε όλα τα πεδία τους).

Η C++ χρησιμοποιεί αντικείμενα και κλάσεις στις οποίες ανήκουν τα αντικείμενα. Όλα τα αντικείμενα που έχουν κοινά χαρακτηριστικά ανήκουν στην ίδια κλάση και κάθε ένα από αυτά λέγεται ότι είναι "στιγμιότυπο" (instance) της κλάσης. Για παράδειγμα, η κλάση «αυτοκίνητο» θεωρείται μια γενική εικόνα του αυτοκινήτου με κάποια χαρακτηριστικά που υπάρχουν σε όλα τα στιγμιότυπα.

Η κλάση «αυτοκίνητο» πιθανώς να ορίζει ότι είναι τετράτροχο, έχει τιμόνι, ταχύτητες, πεντάλ και καθίσματα για τους οδηγούς αλλά δεν ορίζει με σαφήνεια το σχήμα ή τους μηχανισμούς όλων των εξαρτημάτων, ούτε τα χαρακτηριστικά της μηχανής (κυβικά, ίπποι, κύλινδροι, κλπ). Αυτά είναι πρόσθετα χαρακτηριστικά για κάθε αντικείμενο και ορίζονται από κάποια **υποκλάση** της κλάσης «αυτοκίνητο» που θα εξηγηθεί στην συνέχεια.

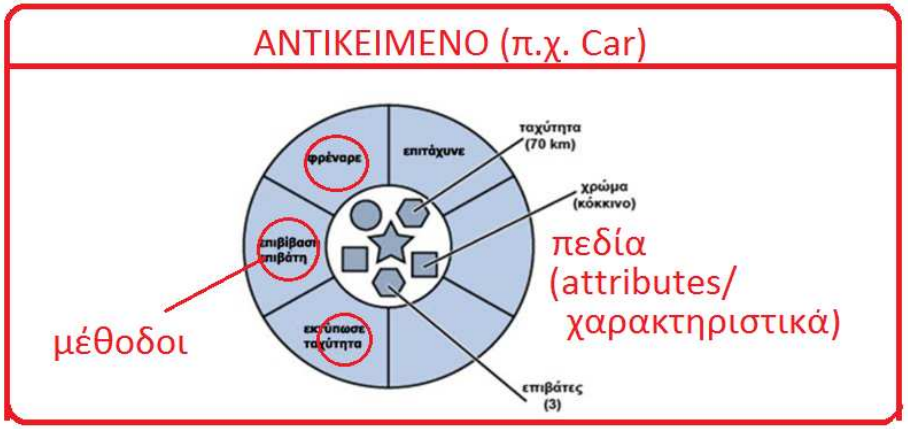
Στη C++, η κλάση δηλώνεται με τη λέξη class.

Στο παράδειγμα του αυτοκινήτου, έστω ότι θέλουμε να δηλώσουμε μια κλάση με το όνομα Car:

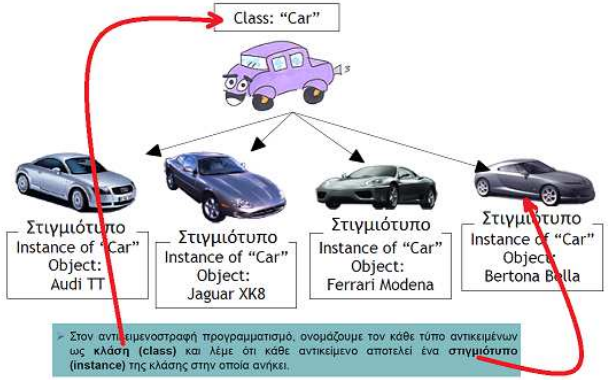
```
class Car
{
        (δεδομένα/μεταβλητές)     // εδώ θα ορίσουμε τις μεταβλητές

        (συναρτήσεις/μέθοδοι)     //εδώ θα ορίσουμε κώδικα
}
```

Τα χαρακτηριστικά του αντικειμένου είναι τα δεδομένα γύρω από τα οποία πρέπει να αναπτύξουμε κάθε πρόγραμμα.



κάθε τύπος αντικειμένου ονομάζεται κλάση/Class



Στο αντικείμενο Car, τα δεδομένα είναι: το χρώμα, η ταχύτητα, η γωνία στρέψης του τιμονιού, η πίεση στα πεντάλ, η θέση του μοχλού ταχυτήτων και άλλα.
Στα προγράμματα μας, αυτά τα δεδομένα θα πρέπει να μοντελοποιηθούν, δηλαδή να γίνει η αντιστοιχία τους σε μεταβλητές τις οποίες μπορούμε να επεξεργαστούμε στο πρόγραμμά μας.

Για την κλάση Car, π.χ. θα έχουμε:

```
class Car
{
        // Γωνία στρέψης του τιμονιού
    float steering_angle;

        // Ποσοστό πατήματος του γκαζιού (0 = καθόλου, 100 = τερματισμένο)
    float gas_pedal;

        // Ποσοστό πατήματος του φρένου (0 = καθόλου, 100 = τερματισμένο)
    float break_pedal;

        // Ποσοστό πατήματος του συμπλέκτη (0 = καθόλου, 100 = τερματισμένο)
    float clutch;

        // θέση της τρέχουσας ταχύτητα (πιθανές τιμές: 0, 1,2,3,4,5,0 = νεκρό, -1 = όπισθεν)
    int gear;

        // μεταβλητές που καθορίζουν επιτάχυνση, ταχύτητα και στροφές κινητήρα
    float acceleration, speed, rpm;
}
```

Φυσικά ο τύπος δεδομένων που επιλέχθηκε για κάθε μεταβλητή εξαρτάται από το είδος της πληροφορίας που θα κρατάει αυτή η μεταβλητή (π.χ. τύπο που θα μπορεί να κρατήσει δεκαδικούς αριθμούς, όπως το float).
Αυτά, συνεπώς είναι τα απαραίτητα δεδομένα <u>υπό την έννοια της κλάσης “Car”.</u>

Μέχρι εδώ δεν είναι τίποτε παραπάνω από μια ομάδα μεταβλητών χωρίς να έχουμε ορίσει τον τρόπο επεξεργασίας τους.

Γι' αυτό είναι απαραίτητη η ύπαρξη του κατάλληλου interface των δεδομένων για τον χρήστη/άνθρωπο/προγραμματιστή.

Αυτό το interface επιτυγχάνεται με την ύπαρξη των μεθόδων της κλάσης (member methods).

Δηλαδή ΜΕΘΟΔΟΙ=ΡΟΥΤΙΝΕΣ=ΥΠΟΡΟΥΤΙΝΕΣ=ΣΥΝΑΡΤΗΣΕΙΣ=ΚΟΜΜΑΤΙΑ_ΚΩΔΙΚΑ=ΠΡΟΓΡΑΜΜΑΤΙΣΤΙΚΟΣ_ΚΩΔΙΚΑΣ που κάνουν πράγματα επάνω στα δεδομένα ή στην «αργκό» της C++ σε ένα στιγμιότυπο!

Οι μέθοδοι (methods) μέσα σε μια κλάση, δεν είναι τίποτε παραπάνω από συναρτήσεις (υπορουτίνες) που προσφέρουν πρόσβαση στα δεδομένα του εκάστοτε αντικειμένου της κλάσης.

Η αλληλεπίδραση του χρήστη (interface) με κάθε αντικείμενο γίνεται μέσω των μεθόδων της κλάσης, οι οποίες καθορίζουν και τον τρόπο χειρισμού των μεταβλητών του αντικειμένου.

Στο Car, κάποιες μέθοδοι θα καθορίζουν με ποιον τρόπο θα στρίβουμε το τιμόνι, θα αυξάνουμε το πάτημα στο γκάζι ή στο φρένο, θα αλλάζουμε ταχύτητες αλλά και πώς θα μπορούμε να γνωρίζουμε την ταχύτητα του αυτοκινήτου, τις στροφές του κινητήρα, κ.λπ.

Διάφορες μέθοδοι για την κλάση «Car» θα μπορούσαν να είναι,

```
// Αλλαγή της γωνία στρέψης του τιμονιού, <relative_angle> μοίρες σε σχέση με την τρέχουσα γωνία.
void turn_wheel(float relative_angle);

// Πάτημα πεντάλ γκαζιού
void press_gas_pedal(float amount);

// Πάτημα πεντάλ φρένου
void press_break_pedal(float amount);
```

```
// Πάτημα πεντάλ συμπλέκτη
void press_clutch_pedal(float amount);

// Αλλαγή της ταχύτητας. Επιστρέφει true αν η αλλαγή ήταν επιτυχής
// ή false αν ήταν ανεπιτυχής (π.χ. από 5 σε όπισθεν).
bool change_gear(int new_gear);

// προβολή της τρέχουσας ταχύτητας, επιτάχυνσης και στροφών του κινητήρα
float get_acceleration();
float get_speed();
float get_rpm();
```

Παρατηρήστε ότι οι παραπάνω μέθοδοι όπου είναι αναγκαίο επιστρέφουν κάποιο αποτέλεσμα,

είτε σαν δεκαδικό αριθμό (float) στην περίπτωση των μεθόδων get_*()
είτε σαν bool στην change_gear().

Παρατηρήστε επίσης ότι ο τύπος void είναι ειδική περίπτωση και δεν επιστρέφει κάποιο αποτέλεσμα. Χρησιμοποιείται όταν μας ενδιαφέρει η εκτέλεση κάποιας διαδικασίας χωρίς όμως να είναι απαραίτητο να γνωρίζουμε το αποτέλεσμά της ή μπορεί να μην χρειάζεται να επιστρέφει τίποτα.

Όλα όμως τα παραπάνω είναι απλώς οι δηλώσεις ΤΩΝ ΜΕΤΑΒΛΗΤΩΝ των μεθόδων, δηλαδή δεν περιέχουν καθόλου προγραμματιστικό κώδικα. <u>Για να είναι ολοκληρωμένος (δηλαδή να έχει νόημα) ο ορισμός μιας μεθόδου θα πρέπει να συνοδεύεται και από την υλοποίησή κάποιου προγραμματιστικού κώδικα (implementation).</u>

Συγκεκριμένα για την προγραμματιστική υλοποίηση της turn_wheel() θα γράφαμε

```
void turn_wheel(float relative_angle)
{
        steering_angle += relative_angle;

        if (steering_angle <= -720.0)
                steering_angle = -720.0;

        if (steering_angle >= 720.0)
                steering_angle = 720.0;
}
```

Με την «προγραμματιστική υλοποίηση» της μεθόδου προσθέσαμε και επιπλέον κώδικα ασφαλείας, ο οποίος δεν αφήνει το τιμόνι να κάνει περισσότερες από 2 στροφές αριστερά ή δεξιά (360° και 360°).

Άρα η μέθοδος μας τώρα έχει **και δηλώσεις πεδίων** (ΜΕΤΑΒΛΗΤΕΣ) αλλά **και κανονικό προγραμματιστικό κώδικα**.